
GUIDE 08

Deploy Agentic AI Apps to Amazon Web Services (AWS)

ECS Fargate, Lambda, EKS, ECR, and Bedrock — step by step

Agentic AI Builder Expert Bootcamp

Batch 4.0 | April 2026

Table of Contents

1. AWS Overview for AI Deployments
2. Prerequisites and CLI Setup
3. Building Production Dockerfiles
4. Amazon ECR (Container Registry) Setup
5. Building and Pushing Images
6. Deploying to ECS Fargate (Recommended)
7. Deploying to AWS Lambda (Serverless)
8. Deploying to Amazon EKS (Kubernetes)
9. Deploying to EC2 Instances
10. Using Amazon Bedrock with Agents
11. Secrets Management with AWS Secrets Manager
12. CI/CD with AWS CodePipeline and GitHub Actions
13. Networking: ALB, API Gateway, and Custom Domains
14. Monitoring with CloudWatch
15. Common Errors and Fixes
16. AWS Deployment Options Comparison

1. AWS Overview for AI Deployments

AWS offers the broadest set of compute services for deploying agentic AI applications, from fully serverless to fully managed Kubernetes.

Key Services for Agent Deployments

- **ECS Fargate:** Serverless container orchestration. No servers to manage. Best for most production agents.
- **AWS Lambda:** Function-as-a-service. Best for lightweight, low-latency agent endpoints (with limitations).
- **Amazon EKS:** Managed Kubernetes for complex multi-agent systems.
- **EC2:** Full virtual machines for maximum control and GPU workloads.
- **Amazon ECR:** Private Docker container registry.
- **Amazon Bedrock:** Managed LLM service with Claude, Llama, Titan, and more.
- **AWS Secrets Manager:** Secure credential storage with automatic rotation.
- **Application Load Balancer (ALB):** HTTP/HTTPS load balancing with path-based routing.

2. Prerequisites and CLI Setup

2.1 Install AWS CLI v2

```
# macOS
brew install awscli

# Windows
msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi

# Linux
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" \
  -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

# Verify
aws --version
```

2.2 Configure Credentials

```
# Configure with IAM user credentials
aws configure
# Enter: Access Key ID, Secret Access Key, Region (ap-southeast-1), Output format (json)

# Or use SSO (recommended for organizations)
aws configure sso
aws sso login --profile my-profile

# Verify
aws sts get-caller-identity
```

2.3 Install Additional Tools

```
# Install ECS CLI (optional but helpful)
brew install amazon-ecs-cli

# Install eksctl for EKS
brew install eksctl

# Install copilot (simplified ECS deployment)
brew install aws/tap/copilot-cli
```

3. Building Production Dockerfiles

3.1 Optimized Multi-Stage Dockerfile

```
# Dockerfile
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install \
    -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY . .

# ECS/Fargate uses a configurable port
ENV PORT=8000
EXPOSE 8000

# Health check for ALB/ECS
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
    CMD curl -f http://localhost:8000/health || exit 1

CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Tip: Unlike Cloud Run (port 8080), AWS ECS does not mandate a specific port. However, always define `HEALTHCHECK` in your Dockerfile -- ECS and ALB rely on it.

3.2 FastAPI Server for AWS

```
# api/main.py
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain_core.messages import HumanMessage
from agent.graph import graph

app = FastAPI(title="LangGraph Agent - AWS")

class ChatRequest(BaseModel):
    message: str
    thread_id: str = "default"
```

```

@app.get("/health")
def health():
    return {"status": "healthy", "platform": "aws-ecs"}

@app.post("/chat")
async def chat(req: ChatRequest):
    try:
        result = graph.invoke({
            "messages": [HumanMessage(content=req.message)]
        })
        return {
            "response": result["messages"][-1].content,
            "steps": len(result["messages"])
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", 8000))
    uvicorn.run(app, host="0.0.0.0", port=port)

```

4. Amazon ECR (Container Registry) Setup

Step 1: Create an ECR Repository

```

aws ecr create-repository \
  --repository-name agentic-ai/langgraph-agent \
  --region ap-southeast-1 \
  --image-scanning-configuration scanOnPush=true \
  --encryption-configuration encryptionType=AES256

# Output includes repositoryUri like:
# 123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/agentic-ai/langgraph-agent

```

Step 2: Authenticate Docker to ECR

```

aws ecr get-login-password --region ap-southeast-1 | \
  docker login --username AWS --password-stdin \
  123456789012.dkr.ecr.ap-southeast-1.amazonaws.com

```

Warning: ECR login tokens expire after 12 hours. Re-run the login command before pushing if your session is stale.

5. Building and Pushing Images

```

# Set variables
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGION=ap-southeast-1
REPO=agentic-ai/langgraph-agent
ECR_URI=${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/${REPO}

```

```
# Build
docker build -t ${ECR_URI}:v1.0 .
docker tag ${ECR_URI}:v1.0 ${ECR_URI}:latest

# Login
aws ecr get-login-password --region ${REGION} | \
    docker login --username AWS --password-stdin \
        ${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com

# Push
docker push ${ECR_URI}:v1.0
docker push ${ECR_URI}:latest

# Verify
aws ecr describe-images \
    --repository-name ${REPO} \
    --region ${REGION}
```

6. Deploying to ECS Fargate (Recommended)

ECS Fargate is the recommended target for production agentic AI. It provides serverless container orchestration without managing EC2 instances.

6.1 Using AWS Copilot (Simplified)

AWS Copilot is the fastest way to deploy. It creates all infrastructure (VPC, ALB, ECS cluster, task definitions) automatically.

```
# Initialize application
copilot app init langgraph-agent

# Initialize service
copilot svc init \
    --name agent-api \
    --svc-type "Load Balanced Web Service" \
    --dockerfile ./Dockerfile \
    --port 8000

# Deploy to test environment
copilot env init --name test --profile default
copilot svc deploy --name agent-api --env test

# Deploy to production
copilot env init --name production --profile default
copilot svc deploy --name agent-api --env production

# View logs
copilot svc logs --name agent-api --env production

# Get the URL
copilot svc show --name agent-api
```

Tip: Copilot generates a manifest file at `copilot/agent-api/manifest.yml`. Edit this for fine-grained control over CPU, memory, scaling, and health checks.

6.2 Copilot Manifest Configuration

```
# copilot/agent-api/manifest.yml
name: agent-api
type: Load Balanced Web Service

image:
  build: Dockerfile
  port: 8000

http:
  path: '/'
  healthcheck:
    path: '/health'
    interval: 30s
    timeout: 10s
    healthy_threshold: 2
    unhealthy_threshold: 3

cpu: 1024      # 1 vCPU
memory: 2048   # 2 GB

count:
  range:
    min: 1
    max: 10
    spot_from: 3 # Use Spot for instances 3+
  cpu_percentage: 70
  requests: 100

variables:
  LANGCHAIN_TRACING_V2: "true"

secrets:
  OPENAI_API_KEY: /copilot/${COPILOT_APPLICATION_NAME}/${COPILOT_ENVIRONMENT_NAME}/secrets/
  openai-key
  LANGCHAIN_API_KEY: /copilot/${COPILOT_APPLICATION_NAME}/${COPILOT_ENVIRONMENT_NAME}/secrets/
  langsmith-key
```

6.3 Manual ECS Fargate Setup (Full Control)

For maximum control, set up ECS components manually.

Step 1: Create an ECS Cluster

```
aws ecs create-cluster \
  --cluster-name agentic-ai-cluster \
  --capacity-providers FARGATE FARGATE_SPOT \
  --default-capacity-provider-strategy \
    capacityProvider=FARGATE,weight=1 \
    capacityProvider=FARGATE_SPOT,weight=3 \
  --region ap-southeast-1
```

Step 2: Create a Task Definition

```
# task-definition.json
{
  "family": "langgraph-agent",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "executionRoleArn": "arn:aws:iam::ACCOUNT:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::ACCOUNT:role/ecsTaskRole",
  "containerDefinitions": [
    {
      "name": "agent",
      "image": "ACCOUNT.dkr.ecr.ap-southeast-1.amazonaws.com/agent-ai/langgraph-agent:v1.0",
      "portMappings": [
        { "containerPort": 8000, "protocol": "tcp" }
      ],
      "environment": [
        { "name": "PORT", "value": "8000" },
        { "name": "LANGCHAIN_TRACING_V2", "value": "true" }
      ],
      "secrets": [
        {
          "name": "OPENAI_API_KEY",
          "valueFrom": "arn:aws:secretsmanager:ap-southeast-1:ACCOUNT:secret:openai-key"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/langgraph-agent",
          "awslogs-region": "ap-southeast-1",
          "awslogs-stream-prefix": "agent"
        }
      },
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3,
        "startPeriod": 15
      }
    }
  ]
}

# Register the task definition
aws ecs register-task-definition \
  --cli-input-json file://task-definition.json
```

Step 3: Create an ECS Service with ALB

```
# Create the service
aws ecs create-service \
  --cluster agent-ai-cluster \
  --service-name langgraph-agent-svc \
```



```

--task-definition langgraph-agent:1 \
--desired-count 2 \
--launch-type FARGATE \
--network-configuration \
    "awsvpcConfiguration={subnets=[subnet-xxx,subnet-yyy],\
    securityGroups=[sg-xxx],assignPublicIp=ENABLED}" \
--load-balancers \
    "targetGroupArn=arn:aws:elasticloadbalancing:..., \
    containerName=agent,containerPort=8000" \
--region ap-southeast-1

```

Step 4: Configure Auto-Scaling

```

# Register scalable target
aws application-autoscaling register-scalable-target \
    --service-namespace ecs \
    --resource-id service/agent-ai-cluster/langgraph-agent-svc \
    --scalable-dimension ecs:service:DesiredCount \
    --min-capacity 1 \
    --max-capacity 20

# Target tracking policy (CPU-based)
aws application-autoscaling put-scaling-policy \
    --service-namespace ecs \
    --resource-id service/agent-ai-cluster/langgraph-agent-svc \
    --scalable-dimension ecs:service:DesiredCount \
    --policy-name agent-cpu-scaling \
    --policy-type TargetTrackingScaling \
    --target-tracking-scaling-policy-configuration \
        "TargetValue=70.0,\
        PredefinedMetricSpecification={PredefinedMetricType=ECSServiceAverageCPUUtilization
}"

```

7. Deploying to AWS Lambda (Serverless)

Lambda is ideal for lightweight agent endpoints with low, sporadic traffic. Note the 15-minute timeout and 10GB memory limits.

7.1 Lambda Container Image

```

# Dockerfile.lambda
FROM public.ecr.aws/lambda/python:3.11

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY agent/ ./agent/
COPY api/ ./api/

# Lambda handler
COPY lambda_handler.py .

```

```

CMD ["lambda_handler.handler"]

# lambda_handler.py
from mangum import Mangum
from api.main import app

# Mangum wraps FastAPI for Lambda
handler = Mangum(app, lifespan="off")

# requirements.txt (add mangum)
mangum==0.18.0
# ... other dependencies

```

7.2 Deploy Lambda

```

# Build and push to ECR
docker build -f Dockerfile.lambda -t ${ECR_URI}-lambda:v1.0 .
docker push ${ECR_URI}-lambda:v1.0

# Create Lambda function
aws lambda create-function \
  --function-name langgraph-agent \
  --package-type Image \
  --code ImageUri=${ECR_URI}-lambda:v1.0 \
  --role arn:aws:iam::ACCOUNT:role/lambda-execution-role \
  --memory-size 2048 \
  --timeout 300 \
  --environment "Variables={\
    OPENAI_API_KEY=sk-xxx,\
    LANGCHAIN_TRACING_V2=true}" \
  --region ap-southeast-1

# Create API Gateway trigger
aws apigatewayv2 create-api \
  --name agent-api \
  --protocol-type HTTP \
  --target arn:aws:lambda:ap-southeast-1:ACCOUNT:function:langgraph-agent

```

Lambda Limitations for Agents

Limitation	Value	Impact on Agents
Max timeout	15 minutes	Complex multi-step chains may timeout
Max memory	10 GB	Sufficient for most agents
Cold start	3-15 seconds (container)	Bad for real-time UX; use provisioned concurrency
Max payload	6 MB (sync)	Large context/responses may exceed
Concurrent executions	1000 (default)	Can request increase

Limitation	Value	Impact on Agents
No persistent state	Stateless per invocation	Need external DB for checkpoints

Warning: For multi-step agents that take > 29 seconds (API Gateway timeout), use Lambda with asynchronous invocation and return results via a callback or polling endpoint.

8. Deploying to Amazon EKS (Kubernetes)

Step 1: Create an EKS Cluster

```
eksctl create cluster \
  --name agentic-ai-cluster \
  --region ap-southeast-1 \
  --nodegroup-name agent-nodes \
  --node-type t3.xlarge \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 5 \
  --managed \
  --with-oidc

# Update kubeconfig
aws eks update-kubeconfig \
  --name agentic-ai-cluster \
  --region ap-southeast-1
```

Step 2: Create Kubernetes Manifests

```
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: langgraph-agent
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: langgraph-agent
  template:
    metadata:
      labels:
        app: langgraph-agent
    spec:
      containers:
        - name: agent
          image: ACCOUNT.dkr.ecr.ap-southeast-1.amazonaws.com/agentic-ai/langgraph-agent:v1.0
          ports:
            - containerPort: 8000
          env:
            - name: OPENAI_API_KEY
```

```

      valueFrom:
        secretKeyRef:
          name: agent-secrets
          key: openai-api-key
    resources:
      requests:
        cpu: "500m"
        memory: "1Gi"
      limits:
        cpu: "2000m"
        memory: "4Gi"
    livenessProbe:
      httpGet:
        path: /health
        port: 8000
      initialDelaySeconds: 15
    readinessProbe:
      httpGet:
        path: /health
        port: 8000
      initialDelaySeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: langgraph-agent-svc
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 8000
  selector:
    app: langgraph-agent

```

Step 3: Deploy

```

kubectl apply -f k8s/deployment.yaml
kubectl get pods
kubectl get svc langgraph-agent-svc

```

9. Deploying to EC2 Instances

```

# Launch an EC2 instance
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafa1f0 \
  --instance-type t3.xlarge \
  --key-name my-key-pair \
  --security-group-ids sg-xxx \
  --subnet-id subnet-xxx \
  --iam-instance-profile Name=ec2-ecr-access \
  --user-data '#!/bin/bash
sudo yum update -y
sudo yum install -y docker
sudo systemctl enable docker

```

```

sudo systemctl start docker
sudo usermod -aG docker ec2-user
# Login to ECR
aws ecr get-login-password --region ap-southeast-1 | \
    docker login --username AWS --password-stdin ACCOUNT.dkr.ecr.ap-southeast-1.amazonaws.com
# Pull and run
docker pull ACCOUNT.dkr.ecr.ap-southeast-1.amazonaws.com/agent-ai/langgraph-agent:v1.0
docker run -d --restart=always -p 8000:8000 \
    -e OPENAI_API_KEY="sk-xxx" \
    ACCOUNT.dkr.ecr.ap-southeast-1.amazonaws.com/agent-ai/langgraph-agent:v1.0'

# SSH in to verify
ssh -i my-key-pair.pem ec2-user@

```

10. Using Amazon Bedrock with Agents

Amazon Bedrock provides managed access to Claude, Llama, Titan, and other foundation models. Here's how to integrate Bedrock with your LangGraph agent.

10.1 Enable Bedrock Model Access

```

# Request model access in AWS Console
# Bedrock > Model access > Request access to:
# - Anthropic Claude Sonnet
# - Anthropic Claude Haiku
# - Meta Llama 3
# - Amazon Titan

```

10.2 LangChain Bedrock Integration

```

# pip install langchain-aws boto3
from langchain_aws import ChatBedrock

# Using Claude via Bedrock
llm = ChatBedrock(
    model_id="anthropic.claude-sonnet-4-20250514-v1:0",
    region_name="ap-southeast-1",
    model_kwargs={
        "temperature": 0,
        "max_tokens": 4096,
    }
)

# In your LangGraph agent
from agent.state import AgentState

def call_model(state: AgentState):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

```

10.3 Bedrock vs Direct API

Dimension	Amazon Bedrock	Direct API (OpenAI/Anthropic)
Billing	Through AWS bill (consolidated)	Separate vendor invoices
Data residency	Data stays in your AWS region	Data sent to vendor's infra
Model selection	Subset of models available	Full model catalog
Pricing	Slightly higher per-token	Standard pricing
IAM integration	Native AWS IAM roles	API key management
VPC connectivity	PrivateLink available	Internet egress required
Compliance	AWS compliance certifications	Vendor-specific

Tip: Use Bedrock if your enterprise requires all data to stay within AWS, consolidated billing, or you already have AWS credits. Use direct API if you need the latest models immediately.

11. Secrets Management

Step 1: Create Secrets

```
# Create a secret
aws secretsmanager create-secret \
  --name openai-key \
  --secret-string "sk-your-openai-key" \
  --region ap-southeast-1

aws secretsmanager create-secret \
  --name langsmith-key \
  --secret-string "ls-your-key" \
  --region ap-southeast-1
```

Step 2: Grant ECS Task Access

```
# The ECS task execution role needs:
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": [
        "arn:aws:secretsmanager:ap-southeast-1:ACCOUNT:secret:openai-key*",
        "arn:aws:secretsmanager:ap-southeast-1:ACCOUNT:secret:langsmith-key*"
      ]
    }
  ]
}
```

Step 3: Reference in Task Definition

```
"secrets": [  
  {  
    "name": "OPENAI_API_KEY",  
    "valueFrom": "arn:aws:secretsmanager:ap-southeast-1:ACCOUNT:secret:openai-key"  
  },  
  {  
    "name": "LANGCHAIN_API_KEY",  
    "valueFrom": "arn:aws:secretsmanager:ap-southeast-1:ACCOUNT:secret:langsmith-key"  
  }  
]
```

12. CI/CD with GitHub Actions

```
# .github/workflows/deploy-aws.yml  
name: Deploy to AWS ECS  
on:  
  push:  
    branches: [main]  
  
env:  
  AWS_REGION: ap-southeast-1  
  ECR_REPOSITORY: agentic-ai/langgraph-agent  
  ECS_SERVICE: langgraph-agent-svc  
  ECS_CLUSTER: agentic-ai-cluster  
  CONTAINER_NAME: agent  
  
jobs:  
  deploy:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
  
      - name: Configure AWS credentials  
        uses: aws-actions/configure-aws-credentials@v4  
        with:  
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }  
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }  
          aws-region: ${ env.AWS_REGION }  
  
      - name: Login to ECR  
        id: login-ecr  
        uses: aws-actions/amazon-ecr-login@v2  
  
      - name: Build, tag, and push  
        id: build  
        env:  
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }  
          IMAGE_TAG: ${ github.sha }  
        run: |  
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .  
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG  
          echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT
```

```
- name: Update ECS service
run: |
  aws ecs update-service \
    --cluster $ECS_CLUSTER \
    --service $ECS_SERVICE \
    --force-new-deployment
```

13. Networking: ALB, API Gateway, and Custom Domains

13.1 Application Load Balancer (for ECS)

```
# Create ALB
aws elbv2 create-load-balancer \
  --name agent-alb \
  --subnets subnet-xxx subnet-yyy \
  --security-groups sg-xxx \
  --type application

# Create target group
aws elbv2 create-target-group \
  --name agent-targets \
  --protocol HTTP \
  --port 8000 \
  --target-type ip \
  --vpc-id vpc-xxx \
  --health-check-path /health \
  --health-check-interval-seconds 30

# Create HTTPS listener
aws elbv2 create-listener \
  --load-balancer-arn ALB_ARN \
  --protocol HTTPS \
  --port 443 \
  --certificates CertificateArn=ACM_CERT_ARN \
  --default-actions Type=forward,TargetGroupArn=TG_ARN
```

13.2 Custom Domain with Route 53

```
# Create A record (alias) pointing to ALB
aws route53 change-resource-record-sets \
  --hosted-zone-id Z12345 \
  --change-batch '{
    "Changes": [{
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "agent.yourdomain.com",
        "Type": "A",
        "AliasTarget": {
          "HostedZoneId": "ALB_HOSTED_ZONE",
          "DNSName": "agent-alb-xxx.ap-southeast-1.elb.amazonaws.com",
          "EvaluateTargetHealth": true
        }
      }
    }]
  }
```



```
}  
}  
}]  
'
```

14. Monitoring with CloudWatch

ECS Metrics (Automatic)

- CPU and memory utilization per task
- Running task count
- Service events and deployment status

Custom Metrics

```
# pip install aws-embedded-metrics  
from aws_embedded_metrics import metric_scope  
  
@metric_scope  
def track_agent_metrics(metrics, **kwargs):  
    metrics.set_namespace("AgenticAI")  
    metrics.put_metric("TokensUsed", token_count, "Count")  
    metrics.put_metric("AgentLatency", latency_ms, "Milliseconds")  
    metrics.put_metric("LLMCost", cost_usd, "None")  
    metrics.set_dimensions(  
        {"Model": "gpt-4o", "Environment": "production"}  
    )
```

CloudWatch Alarms

```
# CPU alarm  
aws cloudwatch put-metric-alarm \  
  --alarm-name agent-high-cpu \  
  --metric-name CPUUtilization \  
  --namespace AWS/ECS \  
  --statistic Average \  
  --period 300 \  
  --threshold 80 \  
  --comparison-operator GreaterThanThreshold \  
  --dimensions Name=ServiceName,Value=langgraph-agent-svc \  
  --evaluation-periods 2 \  
  --alarm-actions arn:aws:sns:ap-southeast-1:ACCOUNT:agent-alerts
```

15. Common Errors and Fixes

Error	Cause	Fix
Task stopped: Essential container exited	App crashed on startup	Check CloudWatch logs; verify env vars and secrets ARNs

Error	Cause	Fix
ECR: denied - requested access denied	Docker not authenticated	Re-run aws ecr get-login-password (tokens expire after 12h)
ECS: unable to pull secrets	Task execution role missing permission	Attach secretsmanager:GetSecretValue policy to execution role
ALB: health check failing	Wrong health check path or port	Verify target group health check matches /health on port 8000
Lambda: Task timed out after 29s	API Gateway timeout (29s max)	Use async invocation or increase to Lambda URL (15 min)
ECS: insufficient memory	Container OOMKilled	Increase task memory; optimize agent state; limit history
Fargate: no ENI available	Subnet IP exhaustion	Use larger subnets; check for IP leaks from stopped tasks
Lambda cold start > 10s	Large container image	Use Lambda Layers; reduce image size; use provisioned concurrency
Bedrock: AccessDeniedException	Model not enabled in console	Go to Bedrock > Model access > Request access for the model
EKS pod: ImagePullBackOff	ECR auth not configured	Create ECR pull secret or use IRSA with ECR permissions

16. AWS Deployment Options Comparison

Feature	ECS Fargate	Lambda	EKS	EC2
Complexity	Medium	Low	High	Low-Medium
Auto-scaling	Yes	Yes (auto)	Yes (HPA)	ASG
Scale to zero	No (min 1 task)	Yes	No	No
Max timeout	Unlimited	15 minutes	Unlimited	Unlimited
GPU support	No	No	Yes	Yes
Cost model	Per vCPU-sec + mem	Per request + duration	Per node	Per instance-hour
Cold start	None (always on)	3-15 seconds	None	None
Best for	Most agent APIs	Low-traffic, light agents	Multi-agent systems	GPU / persistent
Setup time	15 min (Copilot)	10 minutes	30 minutes	10 minutes

Feature	ECS Fargate	Lambda	EKS	EC2
Managed infra	Fully managed	Fully managed	Managed control plane	Self-managed

Recommendation: Use ECS Fargate with AWS Copilot for 80% of agent deployments -- it's the sweet spot of simplicity and production readiness. Use Lambda only for lightweight, infrequent agent calls. EKS for complex multi-agent architectures. EC2 for GPU workloads or persistent background agents.

Bonus: Cross-Cloud Deployment Comparison

Here's how AWS, GCP, and Azure compare for agentic AI deployments:

Capability	AWS	GCP	Azure
Best serverless option	ECS Fargate	Cloud Run	Container Apps
Simplest deployment	Copilot CLI	gcloud run deploy	az containerapp create
Container registry	ECR	Artifact Registry	ACR
Managed Kubernetes	EKS	GKE	AKS
LLM service	Bedrock	Vertex AI	Azure OpenAI
Secrets manager	Secrets Manager	Secret Manager	Key Vault
CI/CD	CodePipeline / GH Actions	Cloud Build	Azure DevOps / GH Actions
Scale to zero	Lambda only	Cloud Run	Container Apps
GPU containers	ECS + EC2 GPU, EKS	GKE + GPU, Compute Engine	AKS + GPU, ACI GPU
Best for	Enterprise with AWS stack	GCP-native or Google AI users	Azure OpenAI / Microsoft stack

Bottom Line: All three clouds can effectively host agentic AI. Choose based on your existing cloud ecosystem, team expertise, and where your LLM provider offers the best integration.